I SEM B.TECH. in CSE
MIDTERM EXAMINATION, SEPT. 2025

SUBJECT: PROBLEM SOLVING USING COMPUTERS (CSE 1071)

Exam Date & Time: 17-Sep-2025 (08:30 AM - 10:00 AM)

Max. Marks: 30 Duration: 90 mins.

Mid-Term Test - SCHEME

Q1. Identify which of the following is valid C expression? (0.5)

1. int my_num = 100,000;
2. **int my_num = 100000;
3. int my num = 1000;
4. int $my_num = 10000

Q2. In C programming language, the size of an array variable is declared by (0.5)

1. Program user
2. **Programmer
3. Software
4. Declared automatically

Q3. Select which of the following is NOT possible with any 2 operators in C? (0.5)

1. Different precedence, same associativity
2. Different precedence, different associativity
3. **Same precedence, different associativity
4. Same precedence, same associativity

Q4. Analyze the C code below and select the correct output. (0.5)

1. ONE
2. ONETWO
3. TWOTHREE
4. **ONETWOTHREE

Q5. Identify the most appropriate output for the below C program. (0.5)

1. C
2. **DAB
3. DC
4. DABC

Q6. Select the most appropriate output for the below C program. (0.5)

1. 1 1 1 2 2 1 2 2
2. **1 1 1 2
3. 1 1 1 2 2 1
4. Infinite loop

Q7. Analyze the C code below and select the correct output. (0.5)

1. **321
2. 32
3. 3210
4. 21

Q8. Identify the output for the below C program. (0.5)

1. 10
2. 15
3. 20
4. **25

Q9. Analyse and fill in the blanks for completing the C code snippet to rotate an array by d elements. (0.5)

Example: for the array 1 2 3 4 5
rotated array by 1 elements: 2 3 4 5 1
rotated array by 3 elements: 4 5 1 2 3

1. **p <= d , i < n - 1 , arr[n - 1] = last;
2. p < d, i < n, arr[n] = last;
3. p >=d, i >n , arr[n+1] = last
4. p >d, i >=n , arr[n] = last

Q10. The given array is arr = {3,4,5,2,1}. The number of swaps in bubble sort and selection sort respectively are _____ (0.5)

1. 5 and 4
2. 3 and 7
3. **7 and 3
4. 2 and 5

Type: DES

Q11. 1. A local library maintains a list of book ID numbers (unique integers) that are not initially sorted. The librarian wants the list to be arranged in ascending order. (4)

Write a C program that:

1. Takes as input a list of book IDs (unsorted).
2. Sorts the IDs using Bubble Sort.
3. Displays the sorted list.

Marking Scheme:

Variable declaration, reading limit and array elements: 1 M
Logic for bubble sort: 1M
Result display for sorting and binary seacrh: 1M

```
#include <stdio.h>
int main() {
    int n, i, j, temp, key;
    int low, high, mid, result = -1;
    printf("Enter number of Book IDs: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d Book IDs: ", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    // Bubble Sort
    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
```

```
    }
    printf("Sorted Book IDs: ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

Q12. Illustrate with an example how a typical C program development environment handles compilation and linking. (3)

Each Step 1M.

1. **You write your C program in a source file (e.g., main.c) using an editor/IDE.**

```
int main()
{
   int result = add(5, 3);
   printf("Result = %d\n", result);
   return 0;
}
```

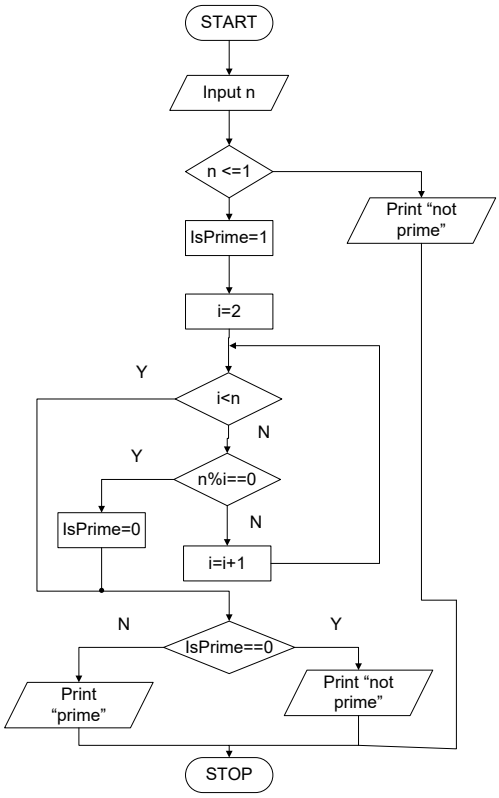
2. **Compilation**
   a. Each .c file is compiled by the **compiler** (like gcc) into an **object file** (.o or .obj).
   b. The compiler translates C code → assembly → machine code.
3. **Linking (Combine Object Files + Libraries)**
   a. The **linker** combines all .o files and required libraries (like libc) into a single **executable**.
4. Execution (Run the Program)

Q13. Write an algorithm and construct a flowchart to check a given number is prime or not. (3)

Algorithm: (1.5M)

Step 1: Start
Step 2: Input an integer n.
Step 3: If n <= 1, then Print "Not Prime" and Stop.
Set a flag variable isPrime = 1
Step 4: for i = 2 to n-1 in steps of 1
If n % i == 0, then Set isPrime = 0
Break the loop.
Step 5: If isPrime == 1, then Print "Prime".
else
Print "Not Prime".
Step 6: Stop

Flowchart(1.5M):



Q14. Write a C program to compute the summation for the following series using *while* loops up to n terms. The variable n is a user input. (3)

$$1 + 3\left(\frac{1}{9}\right) + 12\left(\frac{1}{9}\right)^2 + 48\left(\frac{1}{9}\right)^3 + \cdots.$$

```
#include<stdio.h>
int main() {
float sum=1.0;
int n;
int m=3;
float term=1/9.0;
int t_count =1;
printf("Enter the number of terms you want to add in the series\n");
scanf("%d",&n); (½ Marks)

/*Using while loop*/ (2 Marks)
while(t_count< n)
{
sum += m * term;
m*=4;
term = term *(1/9.0);
t_count++;
}
printf("The sum is:%f", sum); (½ Marks)
}
```

Q15. Using the syntax of the conditional operator, briefly explain its working. Write the equivalent code snippet for the below if statements using conditional operator. (3)

```
if(year%4 == 0) {
if( year%100 == 0) {
if (year%400 == 0)
 printf("%d is a leap year",year);
else
 printf("%d is not a leap year",year);
}else printf("%d is a leap year",year);
}else printf("%d is not a leap year",year);
```

Conditional Operator

condition ? expression1 : expression2

• condition is an expression that is evaluated first.

• If the result of the evaluation of condition is TRUE (nonzero), then expression1 is evaluated and the result of the evaluation becomes the result of the operation.

• If condition is FALSE (zero), then expression2 is evaluated and its result becomes the result of the operation.

Example:

maxValue = (a > b) ? a : b;

Equivalent to:

if (a > b)

maxValue = a;

else
maxValue = b;
[**syntax – 1 mark and brief explanation – 1 mark**]

Equivalent code snippet using conditional operator
(year%4==0) ?
(year%100==0?
(year%400==0 ? printf("The year %d is a leap year",year)
:printf("The year %d is not a leap year",year))
: printf("The year %d is a leap year",year))
: printf("The year %d is not a leap year",year);
[**1 mark**]

Q16. Write a C program to read Two One Dimensional Arrays of same data type (integer type) and merge them into another One Dimensional Array of same type. Accept an integer and delete that element from the merged array. (3)

Sample I/O:

```
Enter the number of elements in the first array: 3
Enter 3 elements for the first array:
1       3       4
Enter the number of elements in the second array: 2
Enter 2 elements for the second array:
22      33
Merged array is:
1 3 4 22 33
Enter the element to delete: 22
Final Array after deletion: 1 3 4 33
```

Marking Scheme:

Variable declaration, reading limit and array elements: ½ M
Logic for merging: 1M
Logic for deletion: 1 M (if all the occurrences deleted full marks; only one occurrence then ½ marks)
Result display: ½ M

```
#include <stdio.h>
int main() {
    int a[10], b[10], merged[20];
    int sizeA, sizeB, i, j,n,delKey,found=0;
    printf("Enter the number of elements in the first array: ");
    scanf("%d", &sizeA);
    printf("Enter %d elements for the first array:\n", sizeA);
    for(i = 0; i < sizeA; i++)
        scanf("%d", &a[i]);

   printf("Enter the number of elements in the second array: ");
    scanf("%d", &sizeB);

    printf("Enter %d elements for the second array:\n", sizeB);
    for(i = 0; i < sizeB; i++)
        scanf("%d", &b[i]);

    // Merging arrays
```

```
    for(i = 0; i < sizeA; i++)
        merged[i] = a[i];
    for(j = 0; j < sizeB; j++)
        merged[i + j] = b[j];

    // Displaying the merged array
    printf("Merged array is:\n");
    for(i = 0; i < sizeA + sizeB; i++)
        printf("%d ", merged[i]);

    printf("\n");
    n=sizeA + sizeB;
    printf("Enter the integer to be deleted: ");
    scanf("%d", &delKey);
    for (i = 0; i < n; i++) {
        if (merged[i] == delKey) {
            found = 1;
            // Shift elements to the left
            for (j = i; j < n - 1; j++) {
                merged[j] = merged[j + 1];
            }
            n--; // Decrement the size of the array
            i--;
        }
    }

    if (!found) {
        printf("Element %d not found in the array.\n", delKey);
    }
    printf("Final Array after deletion:\n");
    for(i = 0; i < n; i++)
        printf("%d ", merged[i]);

    return 0;
}
```

Q17. Analyze the following code and evaluate the expressions. Show all the intermediate steps for the expression evaluation process to arrive at the final results. (2)

```
#include<stdio.h>
void main()
{
int i=3,j=4,k=8;
int z=4*i%5+4*j+5%i*j-3;
int c = (i & j) * 2 + (i | k) - (j ^ k) + (k <= j) * 4;
printf("%d\t%d",z, c);
}
```

<u>For 1st expression (1 MARK)</u>

z=(4∗3)%5+(4∗4)+(5%3)∗4−3
z=(12%5)+16+(2*4)-3
z=2+16+8-3

z=26-3
z=23

For 2nd expression (1 MARK)

c=(011 & 100)*2+(0011|1000)-(0100^1000)+(8<=4)*4

c=(0)*2+(1011)-(1100)+(0)*4
c=0+11-12+0

c=-1

Q18. Write a C program that reads a player's skill level L from the user and prints an appropriate title based on the following conditions using an ***else-if ladder*** statement. (2)

i. L = 5 → "Grandmaster"
ii. L = 4 → "Expert"
iii. L = 3 → "Advanced"
iv. L = 2 → "Intermediate"
v. L = 1 → "Beginner"
vi. For any other value → "Unrated"

```
#include <stdio.h>
int main() {
    int L;
    printf("Enter your skill level (1-5): ");
    scanf("%d", &L);                        0.5 Marks
    if (L == 0) {
        printf("No skill level provided!\n");
    } else if (L < 1 || L > 5) {
        printf("Invalid skill level!\n");
    } else if (L >= 5 && L <= 5) {  // Equivalent to L==5
        printf("Grandmaster\n");                0.5 Marks
    } else if (L >= 4 && L <= 4) {  // Equivalent to L==4
        printf("Expert\n");
    } else if (L >= 3 && L <= 3) {  // Equivalent to L==3
        printf("Advanced\n");                   0.5 Marks
    } else if (L >= 2 && L <= 2) {  // Equivalent to L==2
        printf("Intermediate\n");
    } else if (L >= 1 && L <= 1) {  // Equivalent to L==1
        printf("Beginner\n");
    }
    return 0;
}                                       0.5 Marks
```

Q19. Given an array of integers (duplicates allowed), write a C program to print all the elements that appear only once (unique elements). (2)

Marking Scheme:
Initialization od array: 0.5M
Logic for printing unique elements: 1M
Display : 0.5M

```
#include <stdio.h>
int main() {
    int arr[10],n ;
    int i, j, count;
```

```
    printf("Enter the array limit: ");
    scanf("%d", &n);
    printf("Enter the array elements: ");
    for(i=0;i<n;i++)
        scanf("%d", &arr[i]);

    printf("The array entered: ");
    for(i=0;i<n;i++)
        printf("%d ", arr[i]);

    printf("\nUnique elemennts are: ");
    for (i = 0; i < n; i++) {
        count = 0;
        for (j = 0; j < n; j++) {
            if (arr[i] == arr[j]) {
                count++;
            }
        }
        if (count == 1) {  // appears only once
            printf("%d ", arr[i]);
        }
    }
    return 0;
}
```

END